notebook

March 29, 2022

[1]:
```
import matplotlib.pyplot as plt
import pandas as pd
import nltk
import string
import redivis
import gensim
import gensim.downloader as api
nltk.download('punkt')
nltk.download('maxent_ne_chunker')
nltk.download('words')
nltk.download('averaged_perceptron_tagger')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('stopwords')
nltk.download('word2vec_sample')
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from nltk.text import Text
from nltk.corpus import stopwords
from nltk import FreqDist
from nltk.data import find
```

```
[nltk_data] Downloading package punkt to /home/root/nltk_data…
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package maxent_ne_chunker to
[nltk_data]     /home/root/nltk_data…
[nltk_data]   Unzipping chunkers/maxent_ne_chunker.zip.
[nltk_data] Downloading package words to /home/root/nltk_data…
[nltk_data]   Unzipping corpora/words.zip.
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /home/root/nltk_data…
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger.zip.
[nltk_data] Downloading package wordnet to /home/root/nltk_data…
[nltk_data]   Unzipping corpora/wordnet.zip.
[nltk_data] Downloading package omw-1.4 to /home/root/nltk_data…
[nltk_data]   Unzipping corpora/omw-1.4.zip.
[nltk_data] Downloading package stopwords to /home/root/nltk_data…
[nltk_data]   Unzipping corpora/stopwords.zip.
```

```
[nltk_data] Downloading package word2vec_sample to
[nltk_data]     /home/root/nltk_data…
[nltk_data]   Unzipping models/word2vec_sample.zip.
```

1 Word Embeddings

1.1 Initialize Word2Vec model

[2]:
```
word2vec_sample = str(find('models/word2vec_sample/pruned.word2vec.txt'))
model = gensim.models.KeyedVectors.load_word2vec_format(word2vec_sample,␣
 ↪binary=False)
model.most_similar(positive=['university'], topn = 15)
```

[2]:
```
[('universities', 0.7003918290138245),
 ('faculty', 0.6780906915664673),
 ('undergraduate', 0.6587096452713013),
 ('campus', 0.6434987783432007),
 ('college', 0.638526976108551),
 ('academic', 0.6317198276519775),
 ('professors', 0.6298646926879883),
 ('undergraduates', 0.6149812936782837),
 ('University', 0.6139305233955383),
 ('student', 0.600540041923523),
 ('colleges', 0.582444429397583),
 ('Universities', 0.5718809366226196),
 ('Faculty', 0.5557606816291809),
 ('regents', 0.5493488311767578),
 ('students', 0.546210765838623)]
```

1.2 Initialize GLoVe Model

[3]:
```
glove_model = api.load("glove-wiki-gigaword-50")
glove_model.most_similar(positive=['university'], topn = 15)
```

```
[==================================================] 100.0% 66.0/66.0MB
downloaded
```

[3]:
```
[('college', 0.874463677406311),
 ('harvard', 0.8710561394691467),
 ('yale', 0.8566808104515076),
 ('graduate', 0.8552882671356201),
 ('institute', 0.8483645915985107),
 ('professor', 0.8417032361030579),
 ('school', 0.8261534571647644),
 ('faculty', 0.8257830142974854),
 ('graduated', 0.8143773078918457),
 ('academy', 0.8103663921356201),
```

```
('princeton', 0.8103403449058533),
('doctorate', 0.7973331212997437),
('studied', 0.795906662940979),
('stanford', 0.7906180024147034),
('georgetown', 0.7885624170303345)]
```

2 Import Data

[4]:
```
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', None)
```

[5]:
```
table = redivis.table("ojo_civ_cases:try5")
```

[6]:
```
data = table.to_dataframe()
```

```
  0%|          | 0/762511 [00:00<?, ?it/s]
```

[7]:
```
p = data['plaintiff'].to_frame()
```

[8]:
```
p.describe()
```

[8]:

	plaintiff
count	737153
unique	96613
top	TOWER LOANS
freq	6259

[ ]:
```
p.plaintiff.value_counts()
```

[ ]:
```
p.plaintiff.str.split(expand=True).stack().value_counts()
```

[ ]:
```
for index, row in p.iterrows():
    words = row['plaintiff']
    if(pd.notna(words)):
        word_token = nltk.word_tokenize(words)
        print(list(nltk.bigrams(word_token)))
```

3 Named Entity Recognition

[ ]:
```
for index, row in p.iterrows():
    words = row['plaintiff']
    if(pd.notna(words)):
        word_token = nltk.word_tokenize(words)
        POS_tags = nltk.pos_tag(word_token)
        NameEn = nltk.ne_chunk(POS_tags)
```

```
        print(NameEn)
```

3.1 POS Play

[ ]:
```
lower_data = p.plaintiff.str.lower().to_frame()
lower_data.plaintiff.str.split(expand=True).stack().value_counts()
```

4 Lemmatizing

[ ]:
```
stop_words = set(stopwords.words("english"))
new_stop_words = ["llc", "lp", "inc", "co"]
stop_words.update(new_stop_words)
lemmatizer = WordNetLemmatizer()
lemmatized_words = []
for index, row in lower_data.iterrows():
    words = row['plaintiff']
    if(pd.notna(words)):
        lemmatizing_words = nltk.word_tokenize(words)
        new_lemmatized_words = [lemmatizer.lemmatize(word) for word in␣
 ↪lemmatizing_words if word not in stop_words and word not in string.
 ↪punctuation and word.isalpha()]
        lemmatized_words.append(' '.join(new_lemmatized_words))

lemmatized_frame = pd.DataFrame(list(zip(lemmatized_words)), columns␣
 ↪=['plaintiff'])
lemmatized_frame.plaintiff.str.split(expand=True).stack().value_counts()
```

4.0.1 Histogram on Plaintiff Column

[ ]:
```
p.plaintiff.str.split(expand=True).stack().value_counts()[:25].plot(kind='barh')
```

4.0.2 Lemmatized Histogram of Plaintiff Data

[ ]:
```
plot = lemmatized_frame.plaintiff.str.split(expand=True).stack().
 ↪value_counts()[:55].plot(kind='bar')
fig = plot.get_figure()
fig.Position = [10, 10, 5500, 4000]
fig.savefig('lemmatized_word_count.pdf', format='pdf', bbox_inches='tight')
```

5 Frequency Distribution

[ ]:
```
text = nltk.Text(lemmatized_frame.plaintiff)
frequency_distribution = FreqDist(text)
plot = frequency_distribution.plot()
plot.get_figure().savefig('frequency.pdf', format='pdf', bbox_inches='tight')
```

[ ]:
```
lemmatized_frame.stack().hist()
```

6 More NER

[ ]:
```
named_entities = []
for index, row in p.iterrows():
    words = row['plaintiff']
    entity_labels = []
    if(pd.notna(words)):
        word_token = nltk.word_tokenize(words)
        POS_tags = nltk.pos_tag(word_token)
        ne = nltk.ne_chunk(POS_tags)
        for node in ne:
            if isinstance(node, nltk.Tree):
                if(node.label() != "GPE"):
                    entity_labels.append(node.label())
        lbl = ""
        if len(entity_labels) > 0:
            lbl = max(set(entity_labels), key = entity_labels.count)

        named_entities.append([lbl, words])

ner_frame = pd.DataFrame(named_entities, columns =['Entity', 'plaintiff'])
ner_frame.describe()
```

It sometimes misidentifies organizations :(

[ ]:
```
plot = ner_frame.Entity.str.split(expand=True).stack().value_counts().
  ↪plot(kind='bar')
fig = plot.get_figure()
fig.Position = [10, 10, 5500, 4000]
fig.savefig('ner.pdf', format='pdf', bbox_inches='tight')
```

[ ]:
```
ner_frame.Entity.str.split(expand=True).stack().value_counts().hist()
```

[ ]:
```
plot = ner_frame.Entity.str.split(expand=True).stack().value_counts().plot.pie()
fig = plot.get_figure()
fig.Position = [10, 10, 5500, 4000]
```

```
fig.savefig('ner_pie.pdf', format='pdf', bbox_inches='tight')
```

[ ]:
```
plot = ner_frame.plaintiff.str.split(expand=True).stack().value_counts().plot.
↪pie()
fig = plot.get_figure()
fig.Position = [10, 10, 5500, 4000]
fig.savefig('ner_pie2.pdf', format='pdf', bbox_inches='tight')
```

6.1 Basic Health Filtering

[ ]:
```
named_entities = []
medical_terms = ["health", "medical", "hospital", "clinic"]
for index, row in p.iterrows():
    words = row['plaintiff']
    entity_labels = []
    if(pd.notna(words)):
        word_token = nltk.word_tokenize(words)
        POS_tags = nltk.pos_tag(word_token)
        ne = nltk.ne_chunk(POS_tags)
        for node in ne:
            if isinstance(node, nltk.Tree):
                if(node.label() != "GPE"):
                    entity_labels.append(node.label())
        lbl = "NO_LABEL"
        if len(entity_labels) > 0:
            lbl = max(set(entity_labels), key = entity_labels.count)

        for term in medical_terms:
            if term in words.casefold():
                lbl = "MEDICAL"

        named_entities.append([lbl, words])

ner_frame = pd.DataFrame(named_entities, columns =['Entity', 'plaintiff'])
ner_frame.Entity.value_counts()
```

7 Medical via Embeddings

7.1 Initialize Word2Vec Medical Filters

[9]:
```
medical_terms = ["health", "medical", "hospital", "clinic"]
medical_embeddings = set()
for term in medical_terms:
    similars = model.most_similar(positive=[term], topn = 10)
    for wordembedding in similars:
        medical_embeddings.add(wordembedding[0].lower())
```

```
print(medical_embeddings)
```

```
{'outpatient', 'wellbeing', 'hospitalized', 'medicine', 'nutrition', 'dental',
'health', 'veterinary', 'patient', 'gynecologist', 'pharmacy', 'welfare',
'doctors', 'orthopedic', 'doctor', 'care', 'physician', 'medical',
'immunization', 'clinics', 'education', 'physicians', 'hospital',
'reproductive', 'dispensary', 'ambulance', 'surgical', 'infirmary', 'clinic',
'hospitals'}
```

[10]:
```
lemmatizer = WordNetLemmatizer()
lemmatized_words = set()
for embedding in medical_embeddings:
    lemmatized = lemmatizer.lemmatize(embedding)
    lemmatized_words.add(lemmatized)

medical_embeddings_w2v = lemmatized_words.difference(set(medical_terms))
print(medical_embeddings_w2v)
```

```
{'outpatient', 'immunization', 'wellbeing', 'education', 'hospitalized',
'medicine', 'nutrition', 'dental', 'gynecologist', 'veterinary', 'patient',
'reproductive', 'pharmacy', 'dispensary', 'welfare', 'ambulance', 'orthopedic',
'surgical', 'infirmary', 'doctor', 'care', 'physician'}
```

7.2 Initialize GLoVe Medical Filters

[11]:
```
medical_terms = ["health", "medical", "hospital", "clinic"]
medical_embeddings_glove = set()
for term in medical_terms:
    similars = glove_model.most_similar(positive=[term], topn = 11)
    for wordembedding in similars:
        medical_embeddings_glove.add(wordembedding[0].lower())

print(medical_embeddings_glove)
```

```
{'outpatient', 'specialists', 'hospitalized', 'public', 'medicine', 'nutrition',
'nearby', 'health', 'healthcare', 'nurse', 'welfare', 'pediatric', 'doctors',
'centers', 'environmental', 'care', 'hospice', 'psychiatric', 'medical',
'clinics', 'nursing', 'study', 'education', 'physicians', 'hospital', 'heart',
'poor', 'ambulance', 'aids', 'prevention', 'clinic', 'hospitals'}
```

[12]:
```
lemmatizer = WordNetLemmatizer()
lemmatized_words = set()
for embedding in medical_embeddings_glove:
    lemmatized = lemmatizer.lemmatize(embedding)
    lemmatized_words.add(lemmatized)

medical_embeddings_glove = lemmatized_words.difference(set(medical_terms))
```

```
print(medical_embeddings_glove)
```

```
{'outpatient', 'psychiatric', 'nursing', 'study', 'education', 'hospitalized',
'public', 'medicine', 'nutrition', 'nearby', 'healthcare', 'aid', 'nurse',
'specialist', 'center', 'heart', 'welfare', 'poor', 'ambulance', 'pediatric',
'prevention', 'environmental', 'doctor', 'care', 'hospice', 'physician'}
```

7.3 Combine the two sets of embeddings

7.3.1 Sanitize

[13]:
```
omit_words = ["poor", "care", "nearby", "education", "study", "public", "camp",␣
 ↪"nutrition", "environmental", "hygiene", "veterinary"]

medical_embeddings_w2v = medical_embeddings_w2v.difference(set(omit_words))
medical_embeddings_glove = medical_embeddings_glove.difference(set(omit_words))
```

7.3.2 Combine

[ ]:
```
medical_embeddings.update(medical_embeddings_glove)
```

[14]:
```
medical_embeddings_both = medical_embeddings_glove.
 ↪intersection(medical_embeddings_w2v)
medical_embeddings_both = medical_embeddings_both.difference(set(omit_words))
print(medical_embeddings_both)
```

```
{'outpatient', 'welfare', 'ambulance', 'hospitalized', 'medicine', 'doctor',
'physician'}
```

[ ]:
```
l = list(medical_embeddings_glove)
l.sort()
print(l)
```

7.4 Do the Medical Filtering

[ ]:
```
def determine_label(entity_labels, plaintiff_string):
    for term in medical_terms:
        if term in plaintiff_string.casefold():
            return "MEDICAL"

    for term in medical_embeddings_both:
        if term in plaintiff_string.casefold():
            return "MEDICAL_EMBED_BOTH"

    for term in medical_embeddings_w2v:
        if term in plaintiff_string.casefold():
            return "MEDICAL_EMBED_W2V"
```

```
    for term in medical_embeddings_glove:
        if term in plaintiff_string.casefold():
            return "MEDICAL_EMBED_GLOVE"

    if len(entity_labels) > 0:
        return max(set(entity_labels), key = entity_labels.count)

    return "NO_LABEL"
```

[ ]:
```
named_entities = []
for index, row in p.iterrows():
    words = row['plaintiff']
    entity_labels = []
    if(pd.notna(words)):
        word_token = nltk.word_tokenize(words)
        POS_tags = nltk.pos_tag(word_token)
        ne = nltk.ne_chunk(POS_tags)
        for node in ne:
            if isinstance(node, nltk.Tree):
                if(node.label() != "GPE"):
                    entity_labels.append(node.label())

        lbl = determine_label(entity_labels, words)
        named_entities.append([lbl, words])

ner_frame = pd.DataFrame(named_entities, columns =['Entity', 'plaintiff'])
ner_frame.Entity.value_counts()
```

[ ]:
```
plot = ner_frame.Entity.str.split(expand=True).stack().value_counts().plot.pie()
plt.legend(ner_frame.Entity.value_counts().index.tolist(), loc="upper left")
fig = plot.get_figure()
fig.Position = [10, 10, 5500, 4000]
fig.savefig('ner_pie.pdf', format='pdf', bbox_inches='tight')
```

[ ]:
```
plot = ner_frame.plaintiff.value_counts()[:25].plot(kind='barh')
fig = plot.get_figure()
fig.Position = [10, 10, 5500, 4000]
fig.savefig('ner_plaintiffs.pdf', format='pdf', bbox_inches='tight')
```

8 Double Label Medical Filtering

[15]:
```
def determine_labels(entity_labels, plaintiff_string):
    for term in medical_terms:
        if term in plaintiff_string.casefold():
            return { "primary": "MEDICAL", "secondary": term.upper() }
```

```
    for term in medical_embeddings_both:
        if term in plaintiff_string.casefold():
            return { "primary": "MEDICAL_EMBED_BOTH", "secondary": term.upper()␣
 ↪}

    for term in medical_embeddings_w2v:
        if term in plaintiff_string.casefold():
            return { "primary": "MEDICAL_EMBED_W2V", "secondary": term.upper() }

    for term in medical_embeddings_glove:
        if term in plaintiff_string.casefold():
            return { "primary": "MEDICAL_EMBED_GLOVE", "secondary": term.
 ↪upper() }

    if len(entity_labels) > 0:
        return { "primary": max(set(entity_labels), key = entity_labels.count),␣
 ↪"secondary": "N/A" }

    return { "primary": "NO_LABEL", "secondary": "N/A" }
```

[16]:
```
named_entities = []
for index, row in p.iterrows():
    words = row['plaintiff']
    entity_labels = []
    if(pd.notna(words)):
        word_token = nltk.word_tokenize(words)
        POS_tags = nltk.pos_tag(word_token)
        ne = nltk.ne_chunk(POS_tags)
        for node in ne:
            if isinstance(node, nltk.Tree):
                if(node.label() != "GPE"):
                    entity_labels.append(node.label())

        lbl = determine_labels(entity_labels, words)
        named_entities.append([lbl['primary'], lbl['secondary'], words])

ner_frame = pd.DataFrame(named_entities, columns =['Entity', 'Secondary Label',␣
 ↪'plaintiff'])
ner_frame.Entity.value_counts()
```

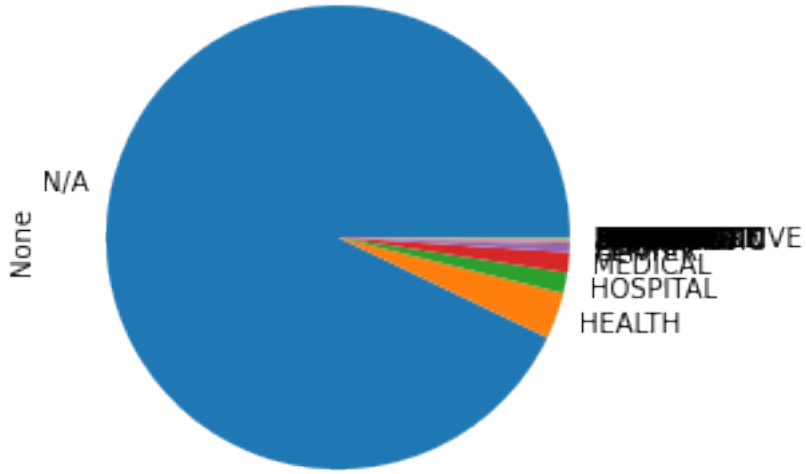
[16]:
```
ORGANIZATION          606235
NO_LABEL               66736
MEDICAL                46110
PERSON                  7637
MEDICAL_EMBED_GLOVE     5344
GSP                     4144
```

```
MEDICAL_EMBED_W2V         496
MEDICAL_EMBED_BOTH        446
FACILITY                    5
Name: Entity, dtype: int64
```

[17]:
```
ner_frame["Secondary Label"].value_counts()
```

[17]:
```
N/A             684757
HEALTH           23838
HOSPITAL         10618
MEDICAL          10442
CENTER            3492
CLINIC            1212
HEART             1167
SPECIALIST         437
PHYSICIAN          315
ORTHOPEDIC         230
DENTAL             155
SURGICAL           104
MEDICINE           103
AID                 80
PEDIATRIC           76
NURSING             71
DOCTOR              20
NURSE               18
AMBULANCE            7
PHARMACY             5
DISPENSARY           1
REPRODUCTIVE         1
WELFARE              1
PREVENTION           1
PSYCHIATRIC          1
HOSPICE              1
Name: Secondary Label, dtype: int64
```

[27]:
```
# Plot the frequency of these labels
plot = ner_frame["Secondary Label"].str.split(expand=True).stack().
 ↪value_counts().plot.pie()
# plt.legend(ner_frame["Secondary Label"].value_counts().index.tolist(),␣
 ↪loc="upper left")
fig = plot.get_figure()
fig.Position = [10, 10, 5500, 4000]
fig.savefig('ojo_ner_pie.pdf', format='pdf', bbox_inches='tight')
```



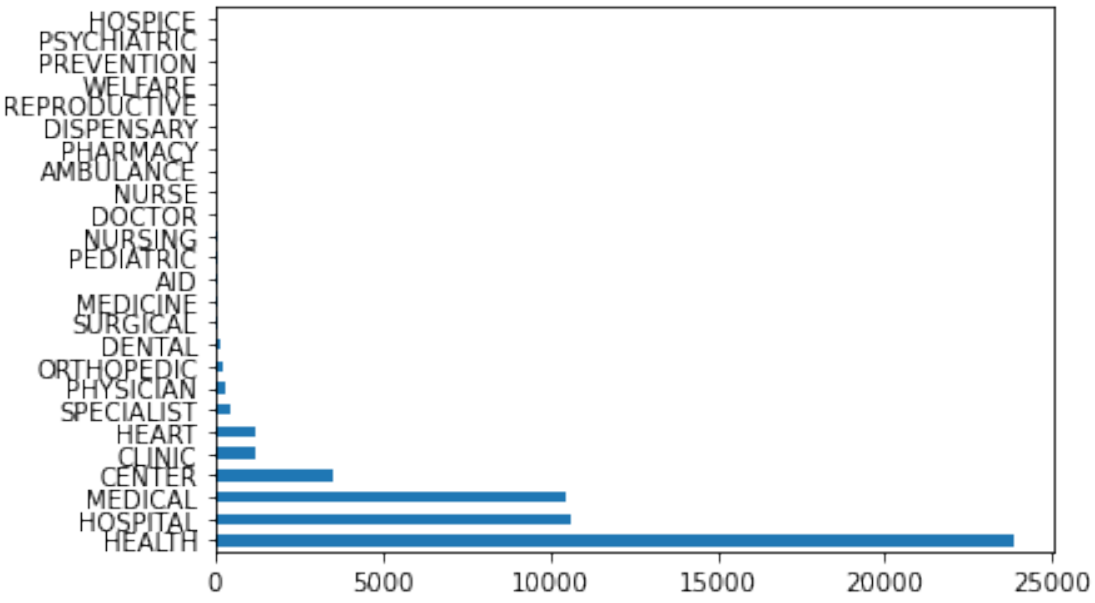
8.0.1 Extract out the medical-related secondary labels

[19]:
```
is_medical = ner_frame["Secondary Label"] != "N/A"
medical = ner_frame[is_medical]
medical["Secondary Label"].value_counts()
```

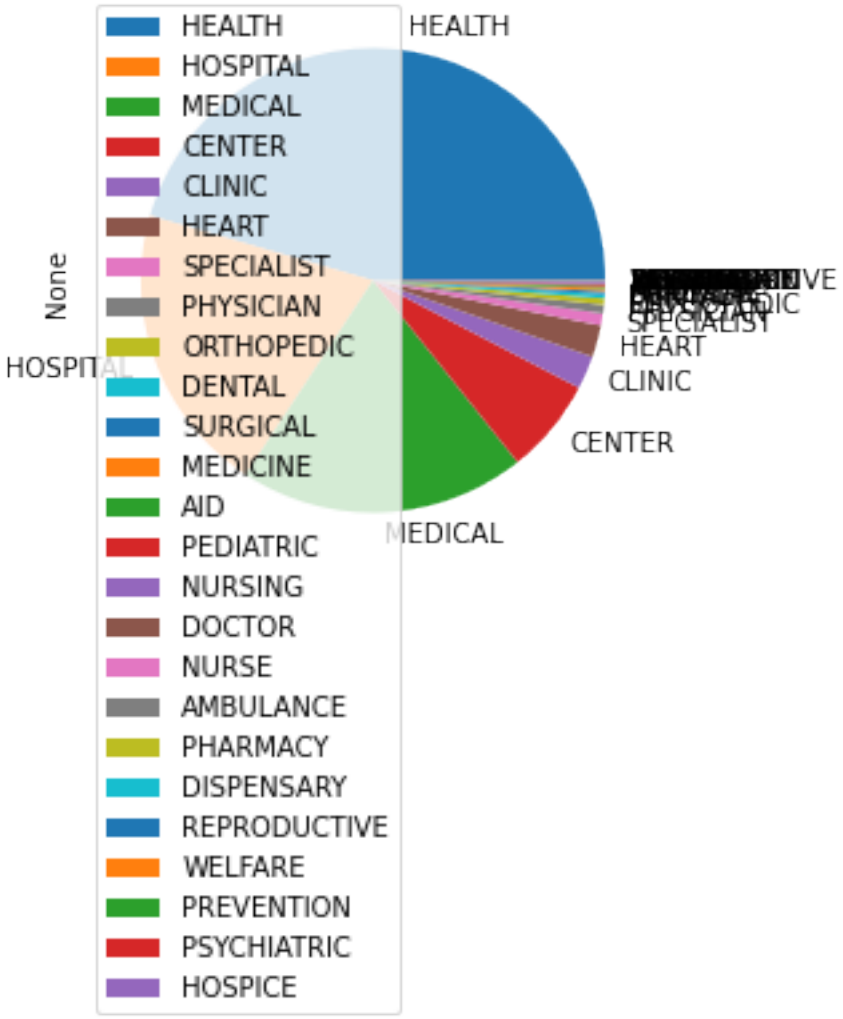
[19]:
```
HEALTH          23838
HOSPITAL        10618
MEDICAL         10442
CENTER           3492
CLINIC           1212
HEART            1167
SPECIALIST        437
PHYSICIAN         315
ORTHOPEDIC        230
DENTAL            155
SURGICAL          104
MEDICINE          103
AID                80
PEDIATRIC          76
NURSING            71
DOCTOR             20
NURSE              18
AMBULANCE           7
PHARMACY            5
DISPENSARY          1
REPRODUCTIVE        1
```

```
WELFARE              1
PREVENTION           1
PSYCHIATRIC          1
HOSPICE              1
Name: Secondary Label, dtype: int64
```

[20]:
```
plot = medical["Secondary Label"].str.split(expand=True).stack().value_counts().
 ↪plot.barh()
fig = plot.get_figure()
fig.Position = [10, 10, 5500, 4000]
fig.savefig('ojo_ner_pie.pdf', format='pdf', bbox_inches='tight')
```



[28]:
```
# Plot the frequency of these labels
plot = medical["Secondary Label"].str.split(expand=True).stack().value_counts().
 ↪plot.pie()
plt.legend(medical["Secondary Label"].value_counts().index.tolist(), loc="upper␣
 ↪left")
fig = plot.get_figure()
fig.Position = [10, 10, 5500, 4000]
fig.savefig('ojo_ner_pie.pdf', format='pdf', bbox_inches='tight')
```


HEALTH
HOSPITAL
MEDICAL
CENTER
CLINIC
HEART
SPECIALIST
PHYSICIAN
ORTHOPEDIC
DENTAL
SURGICAL
MEDICINE
AID
PEDIATRIC
NURSING
DOCTOR
NURSE
AMBULANCE
PHARMACY
DISPENSARY
REPRODUCTIVE
WELFARE
PREVENTION
PSYCHIATRIC
HOSPICE
None
HOSPITAL
HEALTH
HEART
CLINIC
CENTER
MEDICAL